BÁO CÁO THỰC HÀNH LAB 2

Kiểm thử và Kiểm chứng Phần mềm: Hộp đen & Hộp trắng (Module Auth - Xác thực PIN)

Học phần: An toàn phần mềm (CSE703093) | Hệ thống: SecureSys Project | Ngày thực hiện: 11/10/2026

---

1. MỤC TIÊU BÀI THỰC HÀNH

Bài thực hành Lab 2 tập trung vào các phương pháp kiểm thử và đo lường an toàn phần mềm ở mức đơn vị (Unit Testing) và tích hợp (Integration Testing), bao gồm:

1. **Phân biệt hai trường phái kiểm thử kinh điển:**
   - *Kiểm thử hộp đen (Black-box Testing):* Thiết kế ca kiểm thử hoàn toàn dựa trên tài liệu đặc tả chức năng (Specification), không tiếp cận mã nguồn bên trong.
   - *Kiểm thử hộp trắng (White-box Testing):* Đánh giá và thiết kế ca kiểm thử dựa trên cấu trúc bên trong của mã nguồn (các câu lệnh, nhánh rẽ điều kiện và đường dẫn thực thi).
2. **Làm chủ kỹ thuật thiết kế test hộp đen:** Áp dụng *Equivalence Partitioning (EP - Phân vùng tương đương)* và *Boundary Value Analysis (BVA - Phân tích giá trị biên)* để tạo tập test tối ưu bao quát các miền giá trị hợp lệ, không hợp lệ và biên giới hạn.
3. **Đo độ phủ mã thực tế (Code Coverage qua gcov):** Sử dụng trình biên dịch GCC với cờ `--coverage` và công cụ `gcov` để lượng hóa tỷ lệ thực thi dòng lệnh (Statement Coverage) và nhánh rẽ (Branch Coverage).
4. **Nhận thức nghịch lý kiểm thử (Testing Paradox):** Hiểu rõ và chứng minh bằng thực nghiệm lý do vì sao **100% test case hộp đen PASS không đồng nghĩa với 100% code coverage**.
5. **Đánh giá an toàn thứ tự kiểm tra điều kiện:** Phân tích nguy cơ rò rỉ thông tin và tấn công từ chối dịch vụ nếu thay đổi thứ tự ưu tiên kiểm tra trong hàm xác thực.

2. ĐẶC TẢ HÀM CẦN KIỂM THỬ (VALIDATE_PIN)

Hàm mục tiêu thuộc phân hệ xác thực SecureSys (Module Auth) được viết bằng ngôn ngữ C:

```
PinResult validate_pin(const char *pin, int failed_attempts);
```

Các quy tắc nghiệp vụ và ràng buộc an toàn:

- **Độ dài và định dạng:** Mã PIN hợp lệ phải có độ dài chính xác 6 ký tự số (các chữ số từ '0' đến '9'). Trả về `PIN_ERR_WRONG_LENGTH` nếu sai độ dài; trả về `PIN_ERR_NON_DIGIT` nếu chứa ký tự chữ, khoảng trắng hoặc ký tự đặc biệt.
- **Chính sách mã PIN yếu (Weak PIN Policy):** Từ chối các mã PIN có các chữ số tăng liên tiếp (ví dụ: `123456`) hoặc giảm liên tiếp (ví dụ: `654321`). Trả về `PIN_ERR_SEQUENTIAL`. Các trường hợp lặp cặp số (ví dụ: `112233`) không vi phạm quy tắc này và được chấp nhận.

- **Chính sách khóa tài khoản (Account Lockout Policy):** Nếu số lần nhập sai liên tiếp trước đó đạt từ 3 lần trở lên (`failed_attempts >= 3`), hệ thống khóa tài khoản ngay lập tức và trả về `PIN_ERR_ACCOUNT_LOCKED`.
- **Lập trình phòng thủ (Defensive Programming):** Nếu con trỏ chuỗi đầu vào bị NULL, hàm lập tức trả về `PIN_ERR_NULL_INPUT` để ngăn chặn lỗi giải tham chiếu con trỏ rỗng (Null Pointer Dereference Crash).

Bảng mã kết quả (PinResult Enum):

Mã Enum	Giá trị	Tên định danh	Ý nghĩa trạng thái
PIN_OK	0	OK	Mã PIN hợp lệ, xác thực thành công
PIN_ERR_WRONG_LENGTH	1	WRONG_LENGTH	Độ dài khác 6 ký tự (quá ngắn, quá dài, chuỗi rỗng)
PIN_ERR_NON_DIGIT	2	NON_DIGIT	Chứa ký tự không phải chữ số ('0'-'9')
PIN_ERR_SEQUENTIAL	3	SEQUENTIAL_WEAK	Mã PIN yếu (dãy số tăng/giảm liên tiếp)
PIN_ERR_ACCOUNT_LOCKED	4	ACCOUNT_LOCKED	Tài khoản đã bị khóa do nhập sai từ 3 lần trở lên
PIN_ERR_NULL_INPUT	5	NULL_INPUT	Con trỏ chuỗi đầu vào mang giá trị NULL

3. THIẾT KẾ BỘ KIỂM THỬ HỘP ĐEN (EP + BVA)

Bộ test hộp đen được thiết kế độc lập dựa trên đặc tả trước khi xem mã nguồn. Toàn bộ 18 test case được sinh và kiểm thử tự động:

STT	Kỹ thuật áp dụng	Đầu vào PIN	Lần sai	Kỳ vọng	Lý do thiết kế
1	EP-valid	482913	0	OK	Lớp hợp lệ: 6 chữ số, không tuần tự
2	EP-valid	739284	1	OK	Lớp hợp lệ: dưới ngưỡng khóa (1 lần sai)
3	EP-invalid-length	482	0	WRONG_LENGTH	Lớp không hợp lệ: quá ngắn (< 6 ký tự)
4	EP-invalid-length	48291378	0	WRONG_LENGTH	Lớp không hợp lệ: quá dài (> 6 ký tự)
5	EP-invalid-length	(Chuỗi rỗng)	0	WRONG_LENGTH	Lớp không hợp lệ: độ dài = 0
6	BVA-length-boundary	48291	0	WRONG_LENGTH	Biên dưới độ dài: 5 ký tự (thiếu 1)
7	BVA-length-boundary	482913	0	OK	Đúng biên chuẩn: 6 ký tự
8	BVA-length-boundary	4829137	0	WRONG_LENGTH	Biên trên độ dài: 7 ký tự (thừa 1)
9	EP-invalid-charclass	48a913	0	NON_DIGIT	Lớp ký tự sai: chứa chữ cái
10	EP-invalid-charclass	48 913	0	NON_DIGIT	Lớp ký tự sai: chứa khoảng trắng
11	EP-invalid-charclass	-89213	0	NON_DIGIT	Lớp ký tự sai: chứa dấu trừ '-'
12	business-rule	123456	0	SEQUENTIAL_WEAK	Quy tắc nghiệp vụ: dãy tăng liên tiếp
13	business-rule	654321	0	SEQUENTIAL_WEAK	Quy tắc nghiệp vụ: dãy giảm liên tiếp
14	business-rule	112233	0	OK	Quy tắc nghiệp vụ: lặp cặp số hợp lệ
15	BVA-lockout-boundary	482913	2	OK	Biên dưới số lần sai: 2 lần (chưa khóa)
16	BVA-lockout-boundary	482913	3	ACCOUNT_LOCKED	Đúng biên khóa: lần sai thứ 3 -> Khóa
17	BVA-lockout-boundary	482913	4	ACCOUNT_LOCKED	Biên trên số lần sai: > 3 lần -> Vẫn khóa

STT	Kỹ thuật áp dụng	Đầu vào PIN	Lần sai	Kỳ vọng	Lý do thiết kế
18	special-value	NULL	0	NULL_INPUT	Giá trị đặc biệt: con trỏ NULL phòng thủ

Kết quả thực thi kiểm thử hộp đen (Python qua C Runner):

```
python python/black_box_tester.py
Ket qua: 18/18 test PASS (Ty le thanh cong: 100%)
Du lieu chi tiet da luu: dataset/black_box_test_results.csv
```

4. ĐO ĐỘ PHỦ MÃ HỘP TRẮNG VÀ PHÂN TÍCH KHOẢNG TRỐNG KIỂM THỬ

4.1. Quy trình đo độ phủ bằng gcov

Chương trình C được biên dịch bằng GCC với cờ công cụ hóa (Instrumentation flags):

```
gcc -Wall -Wextra -std=c11 --coverage -DPIN_CLI_MAIN -c pin_validator.c -o pin_validator.o
gcc --coverage -o pin_validator_cli pin_validator.o
```

Khi driver CLI chạy file CSV bộ test hộp đen, tệp dữ liệu thực thi `pin_validator.gcda` được tạo ra. Lệnh `gcov pin_validator.c` phân tích và sinh báo cáo chi tiết `pin_validator.c.gcov`.

4.2. Báo cáo độ phủ ban đầu

Chỉ số đo lường	Giá trị ghi nhận
Tổng số dòng có thể thực thi (Executable Lines)	66 dòng
Số dòng đã được chạy qua (Covered Lines)	61 dòng
Tỷ lệ độ phủ dòng lệnh (Statement Coverage)	**92.42%**

4.3. Phân tích các dòng chưa được kiểm thử (Coverage Gaps)

Báo cáo gcov chỉ ra 5 dòng lệnh chưa từng được thực thi (đánh dấu #####):

```
Dong 84:  default: return "UNKNOWN";
Dong 93:  fprintf(stderr, "Cach dung: %s <test_cases.csv>
", argv[0]);
Dong 94:  return 1;
Dong 98:  fprintf(stderr, "[LOI] Khong mo duoc file: %s
", argv[1]);
Dong 99:  return 1;
```

Giải thích bản chất nghịch lý kiểm thử (Testing Paradox):
Tại sao bộ test hộp đen đạt kết quả tuyệt đối **18/18 PASS (100%)** nhưng độ phủ mã chỉ đạt **92.42%**?
- *Nguyên nhân:* Kiểm thử hộp đen chỉ dựa trên tài liệu đặc tả nghiệp vụ của hàm `validate_pin()`. Toàn bộ các luồng nghiệp vụ của hàm đều đã được duyệt qua 100%.
- Tuy nhiên, khi đóng gói thành chương trình hoàn chỉnh, mã nguồn phát sinh thêm tầng giao diện dòng lệnh (CLI Driver) chứa các nhánh xử lý lỗi hệ thống: người dùng không truyền tham số (`argc < 2`), lỗi mở tệp dữ liệu (`fopen() thất bại`), và nhánh phòng ngừa giá trị enum lạ (`default`).
- Các nhánh này nằm ngoài phạm vi đặc tả nghiệp vụ, đòi hỏi phải có **kiểm thử hộp trắng** ở tầng tích hợp mới có thể bao quát toàn diện.

5. BỔ SUNG TEST CASE HỘP TRẮNG NÂNG ĐỘ PHỦ MÃ (COVERAGE BOOST)

5.1. Thiết kế các ca kiểm thử bổ sung

Dựa trên cấu trúc mã nguồn được phân tích qua gcov, chúng tôi thiết kế bổ sung 2 ca kiểm thử hộp trắng chuyên biệt nhắm thẳng vào các nhánh lỗi CLI:

1. **Test case White-box 1 (Thiếu đối số CLI):** Thực thi `pin_validator_cli` không truyền tham số dòng lệnh -> Kích hoạt nhánh `argc < 2`, phủ dòng 93 và 94.
2. **Test case White-box 2 (Tệp dữ liệu không tồn tại):** Thực thi `pin_validator_cli nonexistent_pin_file.csv` -> Kích hoạt nhánh `!fp`, phủ dòng 98 và 99.

5.2. Kết quả độ phủ sau khi bổ sung

Sau khi chạy bổ sung 2 ca kiểm thử trên:

- Số dòng thực thi được tăng từ: **61 / 66** lên **65 / 66 dòng**.
- Tỷ lệ độ phủ mã (Statement Coverage) tăng từ: **92.42% lên 98.48%** (vượt xa ngưỡng yêu cầu tối thiểu 90% của đề bài). Dòng duy nhất chưa chạy là nhánh `default` của hàm chuyển chuỗi (chỉ kích hoạt khi truyền mã enum ngoài phạm vi).

5.3. Minh họa bản vá mã nguồn bổ sung (Patch Diff)

```
--- a/python/white_box_coverage.py
+++ b/python/white_box_coverage.py
@@ -78,6 +78,17 @@
+def run_additional_coverage_tests():
+    # Bo sung test case hop trang de tang do phu ma:
+    # - Test 1: CLI khong truyen doi so (argc < 2) -> phu dong 93, 94.
+    # - Test 2: CLI truyen tep khong ton tai (!fp) -> phu dong 98, 99.
+    cli_name = "pin_validator_cli.exe" if os.name == "nt" else "pin_validator_cli"
+    cli_path = os.path.join(C_DIR, cli_name)
+    subprocess.run([cli_path], cwd=C_DIR, capture_output=True, text=True)
+    subprocess.run([cli_path, "nonexistent_pin_file.csv"], cwd=C_DIR, capture_output=True,
text=True)
```

6. KẾT QUẢ BỘ KIỂM THỬ TỰ ĐỘNG TOÀN DIỆN

Bộ kiểm thử tự động `tests/test_pin_validator.py` được hoàn thiện với 7 test case toàn diện:

STT	Tên Test Case	Mục tiêu kiểm tra	Kết quả
1	test_black_box_suite_all_pass	Xác nhận toàn bộ 18 test case hộp đen đạt chuẩn	PASS
2	test_white_box_coverage_meets_threshold	Xác nhận độ phủ ban đầu đạt ngưỡng >= 90% (92.42%)	PASS
3	test_account_lockout_boundary	Kiểm tra riêng biệt giá trị biên khóa tài khoản (2, 3, 4 lần)	PASS
4	test_cli_negative_cases_boost_coverage	Xác nhận bổ sung test hộp trắng nâng độ phủ lên >= 98%	PASS
5	test_sequential_weak_pin_rules	Kiểm tra quy tắc từ chối PIN yếu tăng/giảm liên tiếp	PASS
6	test_defensive_null_and_empty_handling	Kiểm tra an toàn phòng thủ với con trỏ NULL và chuỗi rỗng	PASS
7	test_failed_attempts_order_security_vulnerability	Chứng minh an toàn của thứ tự kiểm tra failed_attempts	PASS

Lệnh thực thi: `python tests/test_pin_validator.py` --> Kết quả: **7/7 test da PASS (100%)**.

7. GIẢI ĐÁP TOÀN DIỆN CÂU HỎI THẢO LUẬN (TRANG 4 - LAB2.PDF)

Câu hỏi:

"Nếu đảo thứ tự kiểm tra failed_attempts >= 3 xuống sau kiểm tra độ dài PIN, hệ thống có còn an toàn không? Minh họa bằng một test case cụ thể."

Phân tích và Trả lời:

KẾT LUẬN: HỆ THỐNG HOÀN TOÀN KHÔNG CÒN AN TOÀN! Việc đảo thứ tự kiểm tra sẽ tạo ra các lỗ hổng bảo mật nghiêm trọng liên quan đến rò rỉ thông tin trạng thái nội bộ, tấn công từ chối dịch vụ và nguy cơ khai thác bộ nhớ.

1. So sánh thứ tự thực thi:

- **Thứ tự an toàn hiện tại (Nguyên tắc Fail-Fast):**

```
if (pin == NULL) return PIN_ERR_NULL_INPUT;
if (failed_attempts >= 3) return PIN_ERR_ACCOUNT_LOCKED;  // <-- KIỂM TRA KHÓA TRƯỚC
size_t len = strnlen(pin, PIN_LEN + 1);
if (len != PIN_LEN) return PIN_ERR_WRONG_LENGTH;
```

- **Thứ tự bị đảo (Thiết kế có lỗ hổng):**

```
if (pin == NULL) return PIN_ERR_NULL_INPUT;
size_t len = strnlen(pin, PIN_LEN + 1);
if (len != PIN_LEN) return PIN_ERR_WRONG_LENGTH;          // <-- ĐẨY LÊN TRƯỚC!
if (failed_attempts >= 3) return PIN_ERR_ACCOUNT_LOCKED;  // <-- BỊ ĐẨY XUỐNG DƯỚI!
```

2. Minh họa bằng Test Case cụ thể:

- **Đầu vào:** `pin = "123"` (mã PIN sai độ dài, chỉ có 3 ký tự), `failed_attempts = 5` (tài khoản đã bị nhập sai 5 lần, trạng thái thực tế đang bị KHÓA CỨNG).
- **Kỳ vọng an toàn:** Hệ thống bắt buộc phải trả về `PIN_ERR_ACCOUNT_LOCKED` (mã 4). Vì chủ thể đã bị tước quyền truy cập, mọi phân tích dữ liệu đầu vào đều không được phép diễn ra.
- **Thực tế khi bị đảo thứ tự:** Hệ thống trả về `PIN_ERR_WRONG_LENGTH` (mã 1)!

3. Các hệ quả an toàn nghiêm trọng:

1. **Rò rỉ thông tin qua Oracle (Information / Oracle Leakage):** Khi kẻ tấn công gửi mã PIN có độ dài khác 6, hệ thống trả về thông báo "Sai độ dài" thay vì "Tài khoản bị khóa". Kẻ tấn công qua đó biết được hệ thống đang thực hiện bước kiểm tra cú pháp trước khi kiểm tra trạng thái tài khoản. Bằng cách quan sát mã lỗi trả về (hoặc thời gian phản hồi), kẻ tấn công có thể thăm dò (probe) các quy tắc xác thực của hệ thống ngay trên một tài khoản đã bị khóa.
2. **Tấn công từ chối dịch vụ (DoS / Resource Exhaustion):** Một tài khoản bị khóa lẽ ra phải bị ngắt kết nối ngay từ cổng đầu tiên. Việc đưa kiểm tra độ dài và phân tích chuỗi lên trước buộc CPU và bộ nhớ phải duyệt qua chuỗi đầu vào (thậm chí chuỗi rất dài nếu kẻ tấn công gửi payload lớn) trước khi từ chối, gây lãng phí tài nguyên xử lý của máy chủ xác thực.
3. **Nguy cơ kích hoạt lỗi bộ nhớ (Memory Safety Bug):** Trong các hệ thống lớn, nếu các bước tiền xử lý chuỗi phía trước có chứa lỗ hổng (ví dụ lỗi tràn bộ đệm Buffer Overflow trong hàm đo độ dài hoặc chuẩn hóa chuỗi), kẻ tấn công có thể khai thác mã độc thành công trên một tài khoản đã bị khóa từ trước.
4. **Vi phạm nguyên tắc an toàn phần mềm:** Vi phạm nguyên tắc *Fail-Fast* (thất bại ngay tại điểm sớm nhất có thể) và nguyên tắc *Complete Mediation* (mọi quyền truy cập phải được kiểm soát toàn diện trước khi thực hiện tác vụ).

8. TỔNG KẾT BÀI THỰC HÀNH

Bài thực hành Lab 2 đã đạt được toàn bộ mục tiêu với chất lượng cao nhất:

1. Thiết lập thành công bộ kiểm thử hộp đen chuẩn mực 18/18 test case theo EP và BVA, đạt tỷ lệ chính xác 100%.
2. Đo độ phủ mã hộp trắng bằng GCC và gcov, làm sáng tỏ khoảng trống kiểm thử và nâng độ phủ từ 92.42% lên 98.48% nhờ bổ sung các ca kiểm thử tích hợp CLI.
3. Hoàn thiện bộ kiểm thử tự động 7/7 test case bao quát đầy đủ các khía cạnh nghiệp vụ, an toàn phòng thủ và bảo vệ biên.
4. Phân tích sâu sắc và chứng minh bằng thực nghiệm lỗ hổng bảo mật khi vi phạm thứ tự kiểm tra trong thiết kế phần mềm an toàn.